

Module 11: File Allocation Methods

1. Allocation Strategies Overview

Method	Direct Access	External Frag	File Expansion	Typical Implementation
Contiguous	Instant ($O(1)$)	Severe	Very Difficult	CD-ROMs, swap space
Linked	Very Slow ($O(n)$)	Zero	Simple dynamic	FAT16 / FAT32
Indexed	Fast ($O(1)$)	Zero	Simple dynamic	UNIX Inodes, ext4

File Allocation Methods: Contiguous, Linked & Indexed

1. Contiguous Allocation

Directory Entry: `<Start, Length>`
e.g. File "count" → Start: 0, Length: 4

Blk 0

Blk 1

Blk 2

Blk 3

Advantages:

- Fastest sequential & direct access
- Single seek to start of file

Disadvantages:

- External Fragmentation
- File growth problem (Cannot expand)
- Requires compaction

2. Linked Allocation

Directory Entry: `<Start, End>`
Blocks are scattered; contain next ptr

Blk 9 →

Blk 16 →

Blk 1 [-1]

Advantages:

- ZERO External Fragmentation
- Files can grow indefinitely

Disadvantages:

- Direct access extremely slow ($O(n)$)
- Pointer storage overhead (4B / block)
- Single corrupted pointer breaks file!

• FAT variant caches table in RAM

3. Indexed Allocation

Directory Entry: `<Index Block #>`
Dedicated block stores array of pointers

Index Block 19: [9, 16, 1, 10, 25]
Direct pointers to scattered blocks

Advantages:

- Fast Direct Access without fragmentation
- No external fragmentation

Disadvantages:

- Index block overhead for tiny files
- Solved by Multi-level / Combined Inode